



SISTEMAS OPERATIVOS

Simulador de planificación de CPU utilizando el algoritmo FCFS

Karla Angelina Ponzon Bernier

Rafael Elías Cabana Campo

Samuel David Polo Montenegro

Diara Ángel Mosquera Perea

Universidad Cooperativa de Colombia
Facultad de Ingeniería

Programa Ingeniería Software

Santa Marta

Introducción

Los sistemas operativos son clave para administrar los recursos de una computadora, asegurando que los programas funcionen bien y que el procesador se use de forma óptima. Dentro de esto, cómo se decide qué programa se ejecuta primero es súper importante, ya que determina el orden en que las tareas usan la CPU.

Este proyecto consiste en armar un simulador con el que puedas jugar para entender el algoritmo FCFS (el primero que llega, primero que se atiende), hecho en Python y usando Tkinter para que se vea bien. Podrás agregar tareas cuando quieras, ver cómo se ejecutan una tras otra y checar cómo afecta al rendimiento general.

La idea de este trabajo es mostrar de forma sencilla cómo funciona la organización de tareas en un sistema operativo. Así, será más fácil entender cómo trabaja la CPU y cómo se manejan todos los recursos de la computadora.

Planteamiento del problema

En los sistemas operativos modernos es necesario administrar múltiples procesos que compiten constantemente por el uso del procesador. La planificación incorrecta de dichos procesos puede generar tiempos de espera elevados, bajo rendimiento y uso ineficiente de los recursos del sistema.

Muchos estudiantes presentan dificultades para comprender cómo funciona internamente la planificación de procesos debido a que normalmente se estudia únicamente desde la teoría. Por esta razón, surge la necesidad de desarrollar una herramienta visual e interactiva que permita simular el comportamiento de un algoritmo de planificación de CPU.

A partir de esta problemática, se propone el desarrollo de un simulador basado en el algoritmo FCFS que permita observar el orden de ejecución de los procesos, los tiempos de espera, el tiempo de retorno y la administración del recurso CPU.

Justificación

El desarrollo de este proyecto permite aplicar conceptos fundamentales relacionados con los sistemas operativos, especialmente en el área de planificación de procesos y administración de recursos.

La implementación de una simulación interactiva facilita la comprensión del funcionamiento interno de la CPU y permite analizar el comportamiento del sistema operativo frente a diferentes procesos.

Además, el proyecto fortalece habilidades relacionadas con:

- Programación en Python.
- Desarrollo de interfaces gráficas.
- Simulación de algoritmos.
- Gestión de procesos.
- Trabajo colaborativo mediante GitHub.
- Interpretación del rendimiento del sistema.

La herramienta desarrollada también posee un enfoque educativo, ya que permite representar visualmente conceptos teóricos vistos en la asignatura.

Objetivos

Objetivo General

Desarrollar un simulador interactivo del algoritmo de planificación FCFS que permita representar la administración de procesos y el uso de la CPU en un sistema operativo.

Objetivos Específicos

- Implementar el algoritmo FCFS utilizando Python.
- Diseñar una interfaz gráfica interactiva utilizando Tkinter.
- Simular el comportamiento de procesos dentro del sistema.
- Analizar tiempos de espera y retorno.
- Representar visualmente el funcionamiento de la CPU.
- Aplicar conceptos relacionados con sistemas operativos y planificación de procesos.

Marco Teórico

Sistemas Operativos

Un sistema operativo es el software encargado de administrar los recursos de hardware y software de un computador. Entre sus funciones principales se encuentran la gestión de memoria, administración de procesos, control de dispositivos y manejo de archivos.

Procesos

Un proceso es un programa en ejecución que utiliza recursos del sistema operativo. Cada proceso puede encontrarse en diferentes estados como:

- Nuevo
- Listo
- Ejecutando
- Finalizado

CPU

La CPU (Unidad Central de Procesamiento) es el componente encargado de ejecutar instrucciones y procesar la información del sistema.

Planificación de Procesos

La planificación de procesos consiste en determinar qué proceso utilizará la CPU y en qué momento.

Algoritmo FCFS

FCFS (First Come, First Served) es uno de los algoritmos más simples de planificación. Su funcionamiento se basa en ejecutar los procesos según el orden de llegada.

Características principales:

- No es expulsivo.
- Respeta el orden de llegada.
- Es sencillo de implementar.
- Utiliza una cola FIFO.

Desarrollo del proyecto

El proyecto fue desarrollado utilizando Python como lenguaje principal y Tkinter para la interfaz gráfica.

El sistema permite:

- Registrar procesos.
- Asignar tiempo CPU.
- Asignar tiempo de llegada.
- Visualizar la cola de procesos.
- Simular la ejecución paso a paso.
- Mostrar métricas de rendimiento.

Estructura del Sistema

El programa se divide en:

- Clase Proceso.
- Interfaz gráfica.
- Simulación FCFS.
- Sistema de métricas.
- Gestión de eventos.

Clase proceso

La clase Proceso representa cada proceso dentro del sistema.

Atributos principales:

Atributo	Función
Nombre	Identificador del proceso
Tiempo_cpu	Tiempo requerido por la CPU
Llegada	Momento en que entra al sistema
Restante	Tiempo restante de ejecución
Estado	Estado actual del proceso

Capturas del código

```
class Proceso:

    def __init__(self, nombre, tiempo_cpu, llegada):

        self.nombre = nombre
        self.tiempo_cpu = tiempo_cpu
        self.llegada = llegada

        self.restante = tiempo_cpu

        self.estado = "Nuevo"

        self.inicio = None
        self.fin = None

procesos = []
```

Fig1. Clase proceso

```

48 def agregar_proceso():
49
50     nombre = entry_nombre.get()
51
52
53     if (
54         nombre == ""
55         or entry_cpu.get() == ""
56         or entry_llegada.get() == ""
57     ):
58
59         escribir_log("⚠ Debes completar todos los campos")
60
61         return
62
63     try:
64
65         tiempo_cpu = int(entry_cpu.get())
66
67         llegada = int(entry_llegada.get())
68
69     except:
70
71         escribir_log("⚠ Tiempo CPU y llegada deben ser números")
72
73         return
74
75
76     nuevo_proceso = Proceso(
77         nombre,
78         tiempo_cpu,
79         llegada
80     )
81
82     procesos.append(nuevo_proceso)
83
84
85     lista_procesos.insert(
86         tk.END,
87         f"{nombre} | CPU: {tiempo_cpu} | Llegada: {llegada}"
88     )
89
90     escribir_log(f"✅ Proceso {nombre} agregado")
91
92
93     entry_nombre.delete(0, tk.END)
94     entry_cpu.delete(0, tk.END)
95     entry_llegada.delete(0, tk.END)

```

Fig2. Agregar proceso

```

✓ def reiniciar():

    procesos.clear()

    lista_procesos.delete(0, tk.END)

    area_log.delete(1.0, tk.END)

    label_cpu_actual.config(text="CPU ACTUAL: LIBRE")

    escribir_log(" Simulación reiniciada")

```

Fig3. Reiniciar todo


```
def simulacion_fcfs():
    if len(procesos) == 0:
        escribir_log(" No hay procesos")
        return

    tiempo_actual = 0

    cola = []

    proceso_actual = None

    completados = []

    escribir_log("\n===== INICIO FCFS =====\n")

    while len(completados) < len(procesos):
        escribir_log(f"\n Tiempo actual: {tiempo_actual}")
```

Fig4. Simulación FCFS

```
for proceso in procesos:
    if proceso.llegada == tiempo_actual:
        proceso.estado = "Listo"
        cola.append(proceso)
        escribir_log(
            f"{proceso.nombre} llegó al sistema"
        )
```

Fig5. Llegada de procesos

```
if proceso_actual is None and len(cola) > 0:
    proceso_actual = cola.pop(0)
    proceso_actual.estado = "Ejecutando"
    if proceso_actual.inicio is None:
        proceso_actual.inicio = tiempo_actual
    escribir_log(
        f" {proceso_actual.nombre} entra a CPU"
    )
```

Fig6. Toma de procesos

```

if proceso_actual:

    label_cpu_actual.config(
        text=f"CPU ACTUAL: {proceso_actual.nombre}"
    )

    proceso_actual.restante -= 1

    escribir_log(
        f" Ejecutando {proceso_actual.nombre}"
        f" | Restante: {proceso_actual.restante}"
    )

```

Fig7. Ejecutar proceso

```

if proceso_actual.restante == 0:

    proceso_actual.estado = "Finalizado"

    proceso_actual.fin = tiempo_actual + 1

    completados.append(proceso_actual)

    escribir_log(
        f"{proceso_actual.nombre} FINALIZÓ"
    )

    proceso_actual = None

```

```

else:

    label_cpu_actual.config(
        text="CPU ACTUAL: LIBRE"
    )

```

Fig8. Finalización

```

def simulacion_fcfs():
    escribir_log("\n===== RESULTADOS =====\n")

    total_espera = 0
    total_retorno = 0

    for proceso in procesos:
        retorno = proceso.fin - proceso.llegada
        espera = retorno - proceso.tiempo_cpu
        total_espera += espera
        total_retorno += retorno

        escribir_log(f" {proceso.nombre}")
        escribir_log(f"   Llegada: {proceso.llegada}")
        escribir_log(f"   Inicio: {proceso.inicio}")
        escribir_log(f"   Fin: {proceso.fin}")
        escribir_log(f"   Espera: {espera}")
        escribir_log(f"   Retorno: {retorno}\n")

    promedio_espera = total_espera / len(procesos)
    promedio_retorno = total_retorno / len(procesos)

    escribir_log(f" PROMEDIOS")

    escribir_log(
        f"Tiempo promedio de espera: "
        f"{promedio_espera}"
    )

    escribir_log(
        f"Tiempo promedio de retorno: "
        f"{promedio_retorno}"
    )

    label_cpu_actual.config(
        text="CPU ACTUAL: FINALIZADA"
    )

```

Fig9. Resultados finales

```

def iniciar_simulacion():
    hilo = threading.Thread(
        target=simulacion_fcfs
    )

    hilo.start()

```

Fig10. Hilo para evitar bloqueos

```

ventana = tk.Tk()

ventana.title(
    "Simulador de Planificación FCFS"
)

ventana.geometry("850x700")

ventana.config(bg=█ "#1e1e1e")

```

Fig11. Interfaz grafica

```
titulo = tk.Label(  
    ventana,  
    text="SIMULADOR DE PLANIFICACIÓN DE CPU - FCFS",  
    font=("Arial", 18, "bold"),  
    fg="white",  
    bg=■ "#1e1e1e"  
)  
  
titulo.pack(pady=15)
```

Fig12. Titulo

```
label_cpu_actual = tk.Label(  
    ventana,  
    text="CPU ACTUAL: LIBRE",  
    font=("Arial", 14, "bold"),  
    fg=■ "#00ff00",  
    bg=■ "#1e1e1e"  
)  
  
label_cpu_actual.pack(pady=10)
```

Fig13. CPU actual

```

frame_inputs = tk.Frame(
    ventana,
    bg="#1e1e1e"
)

frame_inputs.pack(pady=10)

# Nombre
tk.Label(
    frame_inputs,
    text="Nombre del proceso",
    fg="white",
    bg="#1e1e1e"
).grid(row=0, column=0, padx=10, pady=5)

entry_nombre = tk.Entry(frame_inputs)

entry_nombre.grid(row=0, column=1)

# Tiempo CPU
tk.Label(
    frame_inputs,
    text="Tiempo CPU",
    fg="white",
    bg="#1e1e1e"
).grid(row=1, column=0, padx=10, pady=5)

entry_cpu = tk.Entry(frame_inputs)

entry_cpu.grid(row=1, column=1)

# Tiempo Llegada
tk.Label(
    frame_inputs,
    text="Tiempo de llegada",
    fg="white",
    bg="#1e1e1e"
).grid(row=2, column=0, padx=10, pady=5)

entry_llegada = tk.Entry(frame_inputs)

entry_llegada.grid(row=2, column=1)

```

Fig14. Frame inputs

```

btn_agregar = tk.Button(
    frame_inputs,
    text="Agregar Proceso",
    command=agregar_proceso,
    bg="#3498db",
    fg="white",
    font=("Arial", 10, "bold")
)

btn_agregar.grid(
    row=3,
    column=0,
    columnspan=2,
    pady=10
)

```

Fig15. Botón agregar

```

lista_procesos = tk.Listbox(
    ventana,
    width=70,
    height=8,
    font=("Arial", 10)
)

lista_procesos.pack(pady=10)

```

Fig14. Lista de procesos

```

frame_botones = tk.Frame(
    ventana,
    bg="■ "#1e1e1e"
)

frame_botones.pack(pady=10)

# Botón iniciar
btn_iniciar = tk.Button(
    frame_botones,
    text="Iniciar Simulación",
    command=iniciar_simulacion,
    bg="green",
    fg="white",
    font=("Arial", 11, "bold"),
    padx=10
)

btn_iniciar.grid(row=0, column=0, padx=10)

# Botón reiniciar
btn_reiniciar = tk.Button(
    frame_botones,
    text="Reiniciar",
    command=reiniciar,
    bg="red",
    fg="white",
    font=("Arial", 11, "bold"),
    padx=10
)

btn_reiniciar.grid(row=0, column=1, padx=10)

```

Fig15. Frame botones

```

area_log = tk.Text(
    ventana,
    width=95,
    height=22,
    bg="black",
    fg="■ "#00ff00",
    font=("Consolas", 10)
)

area_log.pack(pady=10)

```

Fig16. Area de logs

```

ventana.mainloop()

```

Fig17. Ejecutar ventana

Interfaz Grafica

La interfaz fue desarrollada con Tkinter y permite al usuario interactuar con el simulador.

Elementos principales:

- Registro de procesos.
- Visualización de CPU actual.
- Cola de procesos.
- Logs de simulación.
- Botones de control.

Simulación

La simulación funciona utilizando una variable de tiempo que avanza paso a paso.

Durante cada ciclo:

1. Llegan nuevos procesos.
2. Los procesos entran a la cola.
3. La CPU selecciona el primero.
4. El proceso consume tiempo CPU.
5. El proceso finaliza.

Explicación del funcionamiento

El usuario registra los procesos ingresando:

- Nombre del proceso.
- Tiempo CPU.
- Tiempo de llegada.

Posteriormente, al iniciar la simulación, el sistema ejecuta los procesos según el algoritmo FCFS.

Ejemplo:

Proceso	Tiempo cpu	Llegada
P1	4	0
P2	3	2
P3	2	4

Funcionamiento:

- P1 entra primero y utiliza la CPU.
- P2 espera en la cola.
- P3 llega posteriormente.
- Los procesos se ejecutan respetando el orden de llegada.

Tecnologías utilizadas

Tecnología	Función
Python	Desarrollo del sistema
Tkinter	Interfaz gráfica
Github	Gestión colaborativa
Vs code	Entorno de desarrollo

Gestión del proyecto

El proyecto fue gestionado mediante GitHub, permitiendo almacenar el código fuente, las evidencias y la documentación.

La utilización del repositorio permitió:

- Trazabilidad de cambios.
- Control de versiones.
- Colaboración entre integrantes.
- Organización del proyecto.

Distribución de roles

Integrante	Rol
Diara Mosquera	Desarrollo de interfaz grafica
Rafael Cabana	Implementación del algoritmo FCFS
Karla Ponzón	Documentación y análisis
Samuel Polo	Evidencias y pruebas

Resultados

El simulador permitió representar correctamente el funcionamiento del algoritmo FCFS.

El sistema mostró:

- Procesos en ejecución.
- Cola de espera.
- Tiempos de espera.
- Tiempos de retorno.
- Estados de procesos.

Además, la interfaz gráfica facilitó la interpretación visual del comportamiento de la CPU.



Fig18. Simulador de CPU-FCFS

Simulador de Planificación FCFS

SIMULADOR DE PLANIFICACIÓN DE CPU - FCFS

CPU ACTUAL: LIBRE

Nombre del proceso

Tiempo CPU

Tiempo de llegada

Agregar Proceso

P1 | CPU: 4 | Llegada: 0
 P2 | CPU: 3 | Llegada: 2
 P3 | CPU: 2 | Llegada: 4

Iniciar Simulación **Reiniciar**

✓ Proceso P1 agregado
 ✓ Proceso P2 agregado
 ✓ Proceso P3 agregado

Fig19. Registro de procesos, tiempo CPU, tiempo0 de llegada

```

===== RESULTADOS =====

P1
Llegada: 0
Inicio: 0
Fin: 4
Espera: 0
Retorno: 4

P2
Llegada: 2

P3
Llegada: 4
Inicio: 7
Fin: 9
Espera: 3
Retorno: 5

PROMEDIOS
Tiempo promedio de espera: 1.6666666666666667
Tiempo promedio de retorno: 4.6666666666666667
  
```

CPU ACTUAL: LIBRE

Nombre del proceso

Tiempo CPU

Tiempo de llegada

Agregar Proceso

Iniciar Simulación **Reiniciar**

Simulación reiniciada

Conclusiones

El desarrollo del simulador de planificación de CPU utilizando el algoritmo FCFS permitió comprender de manera práctica el funcionamiento interno de los sistemas operativos en la administración de procesos y recursos computacionales. A través de la implementación realizada en Python con interfaz gráfica en Tkinter, fue posible representar visualmente cómo los procesos ingresan al sistema, esperan en cola, utilizan la CPU y finalizan su ejecución respetando el orden de llegada. El proyecto facilitó el análisis de conceptos fundamentales como tiempo de espera, tiempo de retorno, estados de procesos y planificación de CPU, permitiendo interpretar el comportamiento del sistema operativo frente a diferentes cargas de trabajo. Además, la simulación interactiva mejoró la comprensión de la relación existente entre los procesos y la administración eficiente de los recursos del sistema. Durante el desarrollo también se fortalecieron habilidades relacionadas con programación, diseño de interfaces gráficas, organización de proyectos y trabajo colaborativo mediante herramientas de control de versiones como GitHub. Finalmente, se concluye que la implementación de simulaciones interactivas representa una estrategia efectiva para el aprendizaje de conceptos complejos de sistemas operativos, ya que permite transformar la teoría en una experiencia visual, dinámica y fácil de interpretar.

